

HearMyHands

Récap de l'intégration web, des optimisations et du déploiement

Pôle Interface – Kaelig Leray

Mai 2026

Ce document résume le travail mené sur la partie web et le déploiement du démonstrateur HearMyHands. Il vient en complément du rapport projet principal, qui détaille la conception scientifique des modèles. Le tout est versionné publiquement sur github.com/nmqx/hearmyhands et accessible en ligne sur hearmyhands.asia.

1. Architecture finale

L'application est servie par un seul processus Python (Flask + Socket.IO sous gunicorn + gevent-websocket), qui héberge à la fois l'interface utilisateur et le moteur d'inférence. Cette fusion a été décidée après mesure : l'aller-retour HTTP entre le serveur web et un service d'inférence séparé ajoutait ~ 40 ms par frame, gaspillés sur boucle locale. Le mode service séparé reste disponible (`USE_HTTP_MODEL=1`) pour un déploiement distribué éventuel.

```
Navigateur --binary JPEG via WebSocket--> Flask/Socket.IO --in-process-->
  InferenceEngine (PyTorch + MediaPipe + MLP + GRU Ocarina) --ack--> Navigateur
```

La logique d'inférence est isolée dans le module `HmH/inference.py` (source de vérité unique), exposée à la fois en in-process et par un wrapper HTTP minimal `HmH/api.py`. Le code reste 100 % open source.

2. Modèles intégrés au runtime

2.1 Pôle Vision (Modèle 1)

Le pôle Vision (responsable : Titouan) a fourni une partie custom basée sur CSPDarknet53 + heatmaps + Soft-Argmax, prédisant 9 points clés du haut du corps en entrée 416×416 . MediaPipe Hands est appelé en complément sur des crops centrés sur les poignets pour extraire les 21 landmarks par main. Les deux briques tournent en parallèle (PyTorch sur GPU/CPU, MediaPipe sur CPU via un `ThreadPoolExecutor`) pour réduire la latence par frame.

2.2 Pôle Traduction (Modèle 2)

Deux modèles cohabitent dans le runtime, sélectionnables via un toggle UI :

- **Mode statique** : MLP $42 \rightarrow 30 \rightarrow 23$ chargé depuis `Poids/{W1,W2,b1,b2}.json`, classification par frame des lettres statiques. Seuil de confiance 0,6, validation par 10 frames stables avant ajout au mot.
- **Mode dynamique** : GRU Ocarina (2 couches, hidden 128, embedding $42 \rightarrow 256 \rightarrow 128$), entraîné par le pôle Traduction (responsable : Killian) sur des séquences de 45 frames. Wrist-relative + scale canonique 640×480 alignés sur `Train.py`. Reconnaît les signes dynamiques (J, Z) que le MLP ne peut pas traiter.

3. Optimisations performances

Profilage initial : 5,1 req/s en service séparé sur `c3-highcpu-8` CPU, avec un goulet à 196 ms/req. Les leviers principaux :

- Transport WebSocket en **JPEG binaire** (au lieu de base64 dataURL), downscale client à 480 px, qualité 0,7 – frames ~ 12 KB au lieu de 80 KB.
- Pipeline non-bloquant côté client : `MAX_IN_FLIGHT=4` avec numéros de séquence, applique uniquement la dernière prédiction.
- Côté serveur : décodage JPEG unique (cv2 / libjpeg-turbo, plus PIL), pré-traitement direct numpy $\rightarrow$ tenseur GPU, `torch.inference_mode`, et parallélisation MediaPipe $\leftrightarrow$ PyTorch.
- Fusion in-process Flask + InferenceEngine pour éliminer l'overhead HTTP local.

Résultat : 44 req/s sustained en in-process sur la même machine (~ 23 ms/req), soit $\sim 8,5\times$ plus rapide. Le client est ensuite cappé à 30 fps (au-delà, le serveur CPU ne suit pas et les frames seraient droppées).

4. Mode apprentissage (style Anki)

Sous-section `/learn` ajoutée pour la dimension pédagogique du projet (**ODD 4**). Une fiche par lettre montre la vidéo modèle (signée par Nestor pour la majorité, Louna pour le Q) à côté de la webcam de l'apprenant. La pipeline d'inférence existante est réutilisée : dès que la prédiction MLP correspond à la lettre cible pendant 5 frames stables au-dessus de 60 % de confiance, une validation visuelle "Bien joué !" s'affiche. L'apprenant note ensuite sa carte parmi *à revoir* / *j'apprends* / *je connais*, et un algorithme de répétition espacée gère la file (intervalles 3, 5, 10, 20, 40, 80 cartes). L'état est persistant via `localStorage`, et une bibliothèque expose toutes les lettres et permet de remettre en révision celles déjà connues.

5. Déploiement

Le démonstrateur est hébergé sur Google Cloud Compute (`c3-highcpu-8` Debian 12 en `europa-west1-b`, IP statique réservée), choisi pour la proximité réseau avec les utilisateurs français (~ 24 ms de RTT depuis Nantes, vs ~ 120 ms quand le serveur était hébergé en `us-central1`). nginx assure le reverse proxy avec les headers WebSocket adéquats, gunicorn fait tourner l'app avec un worker `geventwebsocket` pour Socket.IO. TLS via Let's Encrypt + certbot (auto-renouvellement par timer systemd), avec une seule chaîne couvrant les quatre *server names* : `hearmyhands.asia`, `www.hearmyhands.asia`, `nmqx.lol`, `www.nmqx.lol`. Cloudflare est posé en frontal pour la protection anti-bots, le cache statique et le HTTP/3.

Le service `hnh.service` est piloté par systemd (`Restart=on-failure`) et un workflow de redéploiement tient en une commande : `ssh hnh@vm "cd hearmyhands && git pull && sudo systemctl restart hnh"`.

6. Monitoring et outillage développeur

Un dashboard temps-réel `/monitor` poll `/stats` chaque seconde et affiche CPU total + par cœur, RAM, disque, load average, ainsi que la consommation du worker gunicorn. Implémentation : psutil + un template HTML autonome, dans la même DA glass + gradient. Le mode sombre (toggle lune/soleil dans la navbar, persistant via `localStorage`) couvre l'ensemble du site, par défaut le mode clair.

Le module `HmH/sign_classifier.py` prévoit un chargement gracieux des poids Ocarina : en l'absence du fichier `ocarina_gru_v1.pth`, l'endpoint `/sign_predict` retourne 503 et la webapp désactive silencieusement les prédictions dynamiques. Cela autorise des démos même lorsque la branche temporelle n'est pas embarquée.

7. Liens utiles

- Application live : hearmyhands.asia/translate
- Apprentissage : hearmyhands.asia/learn
- Monitoring : hearmyhands.asia/monitor
- Code source : github.com/nmqx/hearmyhands
- ODD 10 : un.org/sustainabledevelopment/fr/inequality/
- ODD 4 : un.org/sustainabledevelopment/fr/education/
- Laboratoire LS2N : ls2n.fr

Remerciements : à Hélène Pérennou pour le suivi pédagogique, à Matthieu Perreira Da Silva pour l'accompagnement technique côté LS2N, et aux 11 autres membres de l'équipe pour le travail collectif.